
北京邮电大学

《大数据安全》实验报告



题目： 加密数据库的安装与使用

主笔人： 班

学院： 网络空间安全学院

协助

班

班

班

班

加密数据库的安装与使用

一、实验内容

(1) 基础部分：任选一个加密数据库，参照使用手册安装配置系统，演示系统的核心功能。

a.参考实验指导书，完成加密数据 CryptDB (<http://css.csail.mit.edu/cryptdb/>) 的演示实验

b.其他加密数据库，例如 Blind seer (2014)，Arx 2017。

(2) 拓展部分：调研分析与改进性实验

a.对 CryptDB 的安全性进行调研分析。

公开的研究结果，例如

2015_CCS_Inference Attacks on Property-Preserving Encrypted Databases

Seny Kamara. Attacking encrypted database systems, blog post, Outsourced bits, snapshot as of Sept 7, 2015

你自己的研究和发现

b.对 CryptDB 的功能改进进行调研。

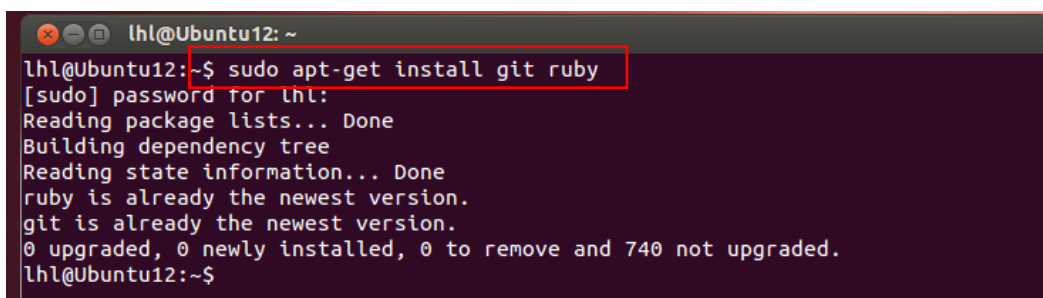
例如, "CryptZip: Squeezing out the Redundancy in Homomorphically Encrypted Backup Data." In Proceedings of the 9th Asia-Pacific Workshop on Systems, pp. 1-8. 2018.

c.同类系统调研分析。也可以设计自己的改进方案。

二、基础部分

(1) 安装 git 和 ruby:

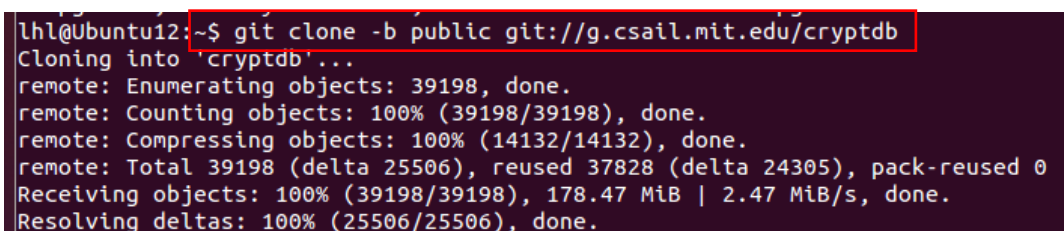
```
sudo apt-get install git ruby
```



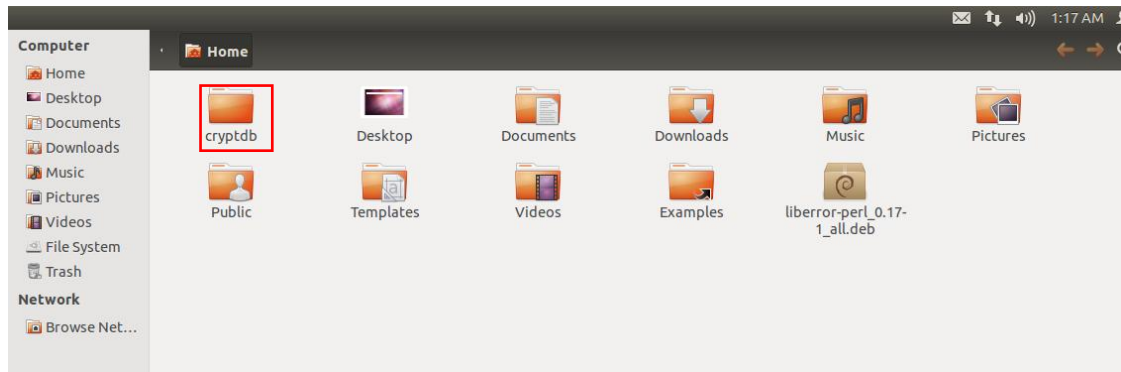
```
lhl@Ubuntu12: ~  
lhl@Ubuntu12:~$ sudo apt-get install git ruby  
[sudo] password for lhl:  
Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
ruby is already the newest version.  
git is already the newest version.  
0 upgraded, 0 newly installed, 0 to remove and 740 not upgraded.  
lhl@Ubuntu12:~$
```

(2) 克隆 CryptDB 代码到用户主目录下面，这里已经提前克隆好了，在用户主目录下：

```
git clone -b public git://g.csail.mit.edu/cryptdb
```

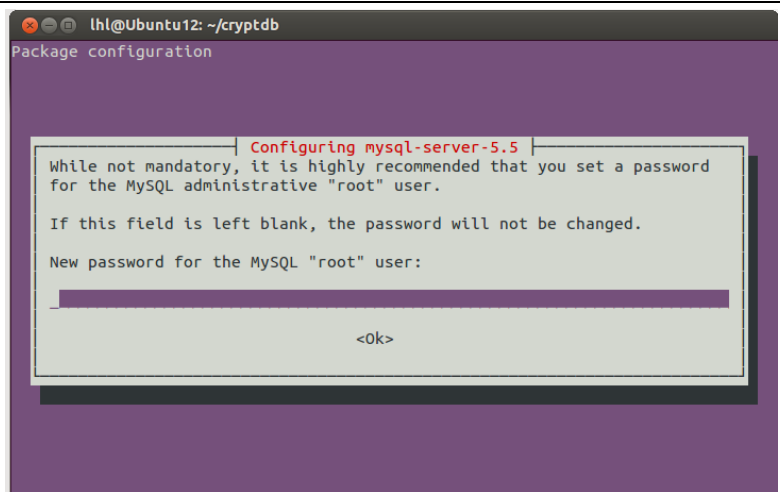


```
lhl@Ubuntu12:~$ git clone -b public git://g.csail.mit.edu/cryptdb  
Cloning into 'cryptdb'...  
remote: Enumerating objects: 39198, done.  
remote: Counting objects: 100% (39198/39198), done.  
remote: Compressing objects: 100% (14132/14132), done.  
remote: Total 39198 (delta 25506), reused 37828 (delta 24305), pack-reused 0  
Receiving objects: 100% (39198/39198), 178.47 MiB | 2.47 MiB/s, done.  
Resolving deltas: 100% (25506/25506), done.
```

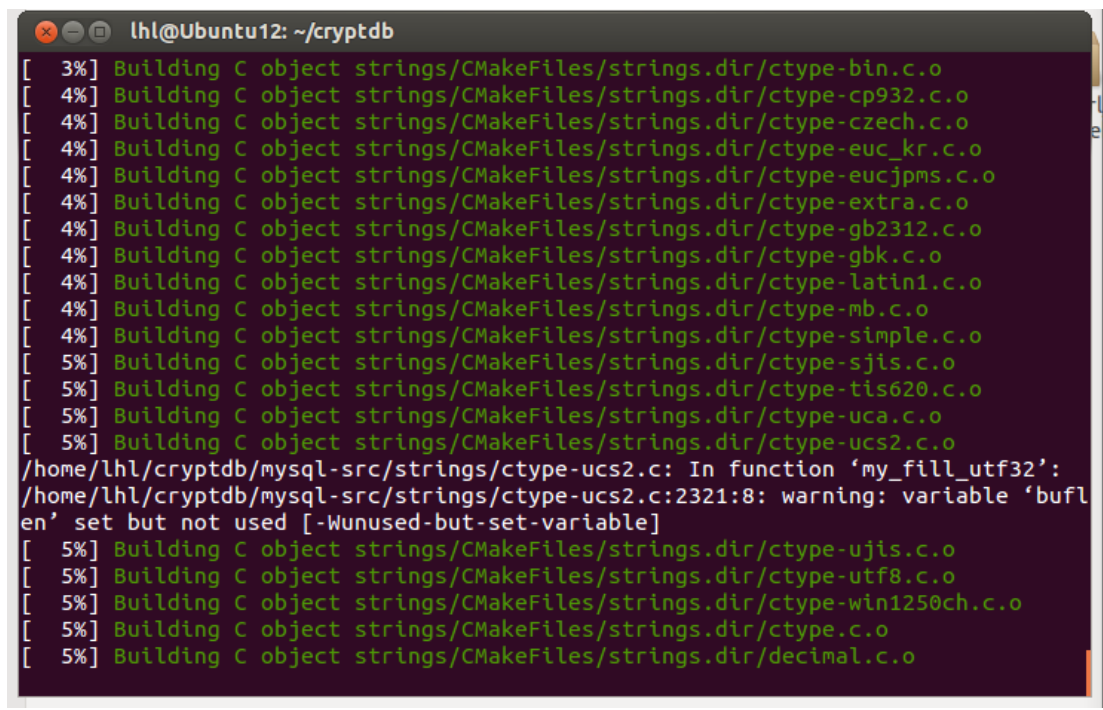


(3) 执行安装脚本：进入到 cryptdb 目录下进行安装。

```
cd cryptdb
sudo ./scripts/install.rb ~/cryptdb
```



安装中会弹出该对话框，需要数据库初始化密码，我使用 letmein。输入后只需等待安装即可，以下是安装过程。



安装完成后结果如图。

```
lhl@Ubuntu12: ~/cryptdb
/paraser/sql_utils.d obj/main/alter_sub_handler.d obj/main/Analysis.d obj/main/cd
b_test.d obj/main/Connect.d obj/main/CryptoHandlers.d obj/main/ddl_handler.d obj
/main/dispatcher.d obj/main/dml_handler.d obj/main/error.d obj/main/metadata_tab
les.d obj/main/rewrite_const.d obj/main/rewrite_ds.d obj/main/rewrite_field.d ob
j/main/rewrite_func.d obj/main/rewrite_main.d obj/main/rewrite_sum.d obj/main/re
write_util.d obj/main/schema.d obj/main/sql_handler.d obj/main/stored_procedures
.d obj/main/Translator.d obj/test/test.d obj/test/TestQueries.d obj/test/test_ut
ils.d obj/util/cryptdb_log.d obj/util/ctr.d obj/util/onions.d obj/util/util.d ob
j/util/version.d obj/udf/edb.d obj/mysqlproxy/ConnectWrapper.d obj/scripts/lockl
ib.d obj/scripts/procs.d
echo "#include <util/version.hh>" > obj/util/version.cc.tmp
( REL="$(git describe --always --long --dirty=+dirty)" ; \
    echo "const char* cryptdb_version_string = \"$REL\";" >> obj/util/vers
ion.cc.tmp )
cmp -s obj/util/version.cc.tmp obj/util/version.cc || mv obj/util/version.cc.tmp
obj/util/version.cc
install -m 644 obj/libedbcrypto.so /usr/lib
install -m 644 obj/libcryptdb.so /usr/lib
install -m 644 obj/libedbutil.so /usr/lib
install -m 644 -g mysql -o mysql obj/udf/edb.so /usr/lib/mysql/plugin
mysql start/running, process 37951
You must do: export EDBDIR=/full/path/to/cryptdb/ before running cryptdb; we rec
ommend putting it into your .bashrc
lhl@Ubuntu12:~/cryptdb$
```

(4)添加环境变量 EDBDIR 到.bashrc: 在 home 目录下打开.bashrc 文件,将 export EDBDIR=/home/lhl/cryptdb/添加到最后。重启后即可使用 CryptDB。

```
sudo vim ~/.bashrc
export EDBDIR=/home/lhl/cryptdb
```

```
lhl@Ubuntu12: ~
# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "${[ $? = 0 ]} && echo terminal || echo
error" "${history|tail -n1|sed -e '\''s/^\s*[0-9]\+\s*//;s/[\;|]\s*alert$//'\''
}'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
    . /etc/bash_completion
fi
export EDBDIR=/home/lhl/cryptdb/

108,1 Bot
```

(5) 本实验使用了三个终端:

终端 1: 用于运行 CryptDB, 在上面显示密文;

终端 2: 用于从代理端口 3306 访问数据库, 显示用户实际操作状态;

终端 3: 用于从正常端口 3307 访问数据库, 显示明文

MySQL 使用本地 3306 端口, CryptDB 使用本地 3307 端口, CryptDB 把 3307 端口的数据处理后通过 3306 端口与 MySQL 交互

(6) 在终端 1 中输入如下内容, 系统返回 started 即为执行成功:

```
/home/lhl/cryptdb/bins/proxy-bin/bin/mysql-proxy
--plugins=proxy --event-threads=4
--max-open-files=1024
--proxy-lua-script=$EDBDIR/mysqlproxy/wrapper.lua
--proxy-address=127.0.0.1:3307
--proxy-backend-addresses=localhost:3306
```

```
lhl@Ubuntu12:~$ /home/lhl/cryptdb/bins/proxy-bin/bin/mysql-proxy --plugins=proxy
--event-threads=4 --max-open-files=1024 --proxy-lua-script=$EDBDIR/mysqlproxy/w
rapper.lua --proxy-address=127.0.0.1:3307 --proxy-backend-addresses=localhost:33
06
2025-10-18 15:04:13: (critical) plugin proxy 0.8.4 started
```

(7) 在终端 2 中输入如下命令，连接到本机 3306 端口的 mysql

```
mysql -u root -p -h 127.0.0.1 -P 3306
```

```
lhl@Ubuntu12:~$ mysql -u root -p -h 127.0.0.1 -P 3306
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 39
Server version: 5.5.62-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

(8) 在终端 3 中输入如下命令，连接到本机 3307 端口的 CryptDB

```
mysql -u root -p -h 127.0.0.1 -P 3307
```

```
lhl@Ubuntu12:~$ mysql -u root -p -h 127.0.0.1 -P 3307
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 40
Server version: 5.5.62-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql>
```

(9) 在终端 3 中查询数据库，此时在终端 1 中显示 CryptDB 查询数据库的结果。

```
show databases
```

```
;
```

```

lhl@Ubuntu12: ~
Server version: 5.5.62-0ubuntu0.12.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases
-> ;
+-----+
| Database |
+-----+
| information_schema |
| cryptdb_udf |
| mysql |
| performance_schema |
| remote_db |
+-----+
5 rows in set (0.01 sec)

mysql>

```

```

lhl@Ubuntu12: ~
251018 15:09:11 InnoDB: Log file /home/lhl/cryptdb//shadow/ib_logfile1 did not
exist: new to be created
InnoDB: Setting log file /home/lhl/cryptdb//shadow/ib_logfile1 size to 5 MB
InnoDB: Database physically writes the file full: wait...
InnoDB: Doublewrite buffer not found: creating new
InnoDB: Doublewrite buffer created
InnoDB: 127 rollback segment(s) active.
InnoDB: Creating foreign key constraint system tables
InnoDB: Foreign key constraint system tables created
251018 15:09:11 InnoDB: Waiting for the background threads to start
=====
QUERY: select @@version_comment limit 1
NEW QUERY: select @@version_comment AS `@@version_comment` limit 1
ENCRYPTED RESULTS:
+-----+
| @@version_comment |
+-----+
| (Ubuntu) |
+-----+
=====
QUERY: show databases
NEW QUERY: show databases

```

(10) 在终端 3 中创建数据库名称为 test3；此时在终端 1 中显示 CryptDB 创建数据库 test3 的结果。

```
create database test3;
```

```

mysql> create database test3;
Query OK, 1 row affected (0.04 sec)

mysql>

```

```
=====
QUERY: create database test3
NEW QUERY: create database `test3`
ENCRYPTED RESULTS:
+
|
+
+
+
ENCRYPTED RESULTS:
+
|
+
+
+
|
```

(11) 在终端 3 中打开数据库 test3; 此时在终端 1 中显示 CryptDB 打开数据库的结果。

```
use test3;
```

```
mysql> use test3;
Database changed
mysql>
```

```
QUERY: show tables
NEW QUERY: show tables
ENCRYPTED RESULTS:
+-----+
|Tables_in_test3|
+-----+
+-----+
```

(12) 在终端 3 中新建表 users。

```
create table users(id int(2) not null primary key auto_increment,username
varchar(40),password varchar(40));
```

此时在终端 1 中显示 CryptDB 新建表 users 的结果。

```
mysql> create table users(id int(2) not null primary key auto_increment,username
varchar(40),password varchar(40));
Query OK, 0 rows affected (0.10 sec)
```

```
mysql> |
```

```
=====
QUERY: create table users(id int(2) not null primary key auto_increment,username
varchar(40),password varchar(40))
NEW QUERY: create table table_YSAGHVXTC (AFIJWXGTREoPLAIN INT(2) unsigned not n
ull auto_increment, LHXSREKCGeEq VARBINARY(80), ZGMIRIFYDwoOrder BIGINT unsigne
d, cdb_saltQPWQVDNHBI BIGINT(8) unsigned, JGQIZIMLKSoEq VARBINARY(80), RMYDUQOBQ
RoOrder BIGINT unsigned, cdb_saltCTOALLGBTH BIGINT(8) unsigned, PRIMARY KEY inde
x_929285797076599327 (AFIJWXGTREoPLAIN)) AUTO_INCREMENT=0 ENGINE=InnoDB
```

```
ENCRYPTED RESULTS:
```

```
+
|
+
+
+
ENCRYPTED RESULTS:
```

```
+
|
+
+
+
|
```


(13) 在终端 3 中的 users 表中增加数据。此时在终端 1 中显示 CryptDB 的 users 表增加数据的结果。

```
insert into users(username,password) values("lihaolun","123456");
```

```
mysql> insert into users(username,password) values("lihaolun","123456");
Query OK, 1 row affected (0.03 sec)
```

```
mysql> 
```

```
=====
QUERY: insert into users(username,password) values("lihaolun","123456")
NEW QUERY: insert into `test3`.`table_YSAGHVKXTC` (`test3`.`table_YSAGHVKXTC`.`L
HXSREKCGEoEq`, `test3`.`table_YSAGHVKXTC`.`ZGMIRIFYDWOOrder`, `test3`.`table_YSA
GHVKXTC`.`cdb_saltQPWQVDNHBI`, `test3`.`table_YSAGHVKXTC`.`JGQIZIMLKSoEq`, `test
3`.`table_YSAGHVKXTC`.`RMYDUQOBQRoOrder`, `test3`.`table_YSAGHVKXTC`.`cdb_saltCT
OALLGBTH`, `test3`.`table_YSAGHVKXTC`.`AFIJWXGTREoPLAIN`) values ('?????'?????S
x?^?]o??G??1{qy?}?????NC??T??=????h?', 16365535026019559897, 7934315272160556049
, 'C<?D???Fg?d?L????N??????_?a?2?'?\0?M1?o??qM?:??', 8107056702006051512, 106
39242433418613440, '\0\')
```

ENCRYPTED RESULTS:

```
+
|
+
+
|
```

(14) 在终端 3 中查询表 users 中的记录,此时在终端 1 中显示 CryptDB 的 users 表查询记录的结果。

```
select * from users;
```

```
mysql> select * from users;
```

```
+-----+-----+-----+
| id    | username | password |
+-----+-----+-----+
| 1     | lihaolun | 123456   |
+-----+-----+-----+
1 row in set (0.02 sec)
```

```
mysql> 
```

```
=====
QUERY: select * from users
NEW QUERY: select `test3`.`table_YSAGHVKXTC`.`AFIJWXGTREoPLAIN`, `test3`.`table_Y
SAGHVKXTC`.`LHXSREKCGEoEq`, `test3`.`table_YSAGHVKXTC`.`JGQIZIMLKSoEq`, `test3`.`t
able_YSAGHVKXTC`.`cdb_saltCTOALLGBTH` from `test3`.`table_YSAGHVKXTC`
ENCRYPTED RESULTS:
```

```
+-----+-----+-----+-----+
|AFIJWXGTREoPLAIN |LHXSREKCGEoEq |JGQIZIMLKSoEq |cdb_saltCTOALLGB
TH |
+-----+-----+-----+-----+
|1 |z??_S?????S\??Lw$e??????????3??r|C<?D???Fg?d?L????N??????
?a?2?'???M1?o??qM?:??|10639242433418613440|
+-----+-----+-----+-----+
|
```


(15) 在终端 1 中发现新创建的 users 表在 Mysql 中储存的名字为 table_YSAGHVKXTC，在终端 2 中查询以此命名的表，发现储存数据为密文，表明数据在 Mysql 端是加密的。

```
select * from table_YSAGHVKXTC;
```

```
mysql> select * from table_YSAGHVKXTC;
+-----+-----+-----+-----+
| AFIJWXGTREoPLAIN | LHXsREKCGEoEq | ZGMIRIFYDWoOrder | c |
| db_saltQPWQVDNHBI | JGQIZIMLKSoEq | RMYDUQOB |
| QRoOrder | cdb_saltCTOALLGBTH |
+-----+-----+-----+-----+
| 1 | z | S | $ | Lw$e | 3 | 163655350260195 |
59897 | 7934315272160556049 | C< | Fg | L | 7 | a | 2 | 2 | M | o | q | M |
| 8107056702006051512 | 10639242433418613440 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
```

(16) 修改 CryptDB 密码，更换 letmein 为其他密码即可。

```
sudo xdg-open /home/lhl/cryptdb/mysqlproxy/wrapper.lua
```

```
-- Use this instead of connect_server(), to get server name
dprint("Connected " .. proxy.connection.client.src.name)
CryptDB.connect(proxy.connection.client.src.name,
    proxy.connection.server.dst.address,
    proxy.connection.server.dst.port,
    os.getenv("CRYPTDB_USER") or "root",
    os.getenv("CRYPTDB_PASS") or "letmein",|
    os.getenv("CRYPTDB_SHADOW") or os.getenv("EDBDIR").."/
shadow")
```

(17) 修改 mysql 密码，输入如下语句并按提示输入当前密码，要更改的密码，并确认密码。

```
mysqladmin -u root -p password
```

```
lhl@ubuntu12:~$ mysqladmin -u root -p password
Enter password:
New password:
Confirm new password:
```

三、拓展部分

3.1 对 CryptDB 的安全性进行调研分析

阅读 Inference Attacks on Property-Preserving Encrypted Databases

(1) 论文主题与研究背景

文章评估基于 property-preserving encryption(PPE) (主要是 Deterministic Encryption(DTE)与 Order-Preserving Encryption(OPE)) 构建的加密数据库

(CryptDB 类设计) 在“实际/具体”场景下对抗推断/统计攻击的安全性, 提出并实证化若干有效的推断攻击。

CryptDB 等系统通过泄露“属性”来保持查询表现, 从而允许在加密上执行许多 SQL 操作。但这种“属性泄露”成为推断攻击的入口, 作者系统化地构造攻击并在真实医疗数据上测试其效果以衡量风险。

(2) 威胁模型

对手能力: 比传统论文里常假定的半诚实服务器还弱, 仅拥有密文数据库 (ciphertext-only) 且不能看到/操控查询 (但能访问到“steady-state”已被“剥洋葱”到支持查询所需层的密文列)。对手还可利用公开的/可得的辅助数据 (如州级住院公开数据、历史库、应用文档等)。

攻击目标: 既包括 individual attacks (恢复单条记录的具体字段), 也包括 aggregate attacks (统计信息或整库分布)。对 EDB 来说, 恢复单个单元格就被视为成功。

(3) 四类主要攻击 (方法与要点)

作者研究并实现了 4 种攻击: 其中两种是已知的 (frequency、sorting), 两种为新提出的 (p-optimization、cumulative)。实现上大量利用直方图/累积分布/排列匹配与赋值问题 (Hungarian 算法/LSAP)。

a. Frequency analysis (针对 DTE)

思路: 对密文列统计频率 (histogram), 用辅助数据做相同统计, 把第 i 频率的密文映射到第 i 频率的明文 (可按频率排序并逐一匹配)。实现非常简单但对许多现实数据非常有效。

DTE (Deterministic Encryption) 使得: $mi = mj \Rightarrow ci = cj$ 因此密文频率分布与明文频率一致。

攻击公式: $fC(ck) =$ 密文第 k 类的频率; $fP(pk) =$ 辅助数据中明文第 k 类频率

则攻击通过排序求映射: $\text{sort}(fC) \leftrightarrow \text{sort}(fP)$

最终输出映射: $ck \rightarrow pk$

适用于性别、死亡风险、疾病严重程度等稳定分布 $\rightarrow$ 恢复率可达 80 - 100%。

b. p-optimization (针对 DTE, 作者提出)

思路: 把映射问题视为寻找一个置换, 使密文直方图 ψ 与置换后的辅助数据直方图 π 的 ℓ_p 距离最小。对于 $p=1$ 可转为线性和分配问题 (LSAP), 用 Hungarian 算法求解; 对于 $p=2,3$ 作者也给出等价的 LSAP 形式 (因此多项式可解)。

优势: 除了给出映射外还能输出“代价”, 可用于后续判断某列是否为目标列 (即攻击可定位隐藏列)。

构造代价矩阵: $\text{Cost}(i, j) = |f_C(c_i) - f_P(p_j)|^p$

求最优匹配使用 Hungarian Algorithm (匈牙利算法) 求:

$$\pi^* = \arg \min_{\pi} \sum_i \text{Cost}(i, \pi(i))$$

$p=2$ 或 $p=3$ 时攻击效果最好等价于在频率空间上求最小代价匹配论文指出表现与频率排序一致甚至更好

c.Sorting attack（针对 OPE，稠密列）

思路：如果 OPE 列是 dense（列中包含了属性空间中几乎所有可能值，即 density=1），排序密文并把排名一一对应到明文空间的排序即可完全恢复（不需要任何辅助数据）。对稠密列几乎是完美破解。

OPE 保留排序结构： $m_i < m_j \Rightarrow c_i < c_j$

若列“足够密”（例如年龄 0~99 几乎全出现）： $\text{rank}(ck) = \text{rank}(mk)$

直接排序对齐即可恢复： $c(1), c(2), \dots, c(n) \leftrightarrow m(1), m(2), \dots, m(n)$

论文实验：年龄（Age）、住院天数（LOS）可恢复 95 - 100%。

d.Cumulative attack（针对 OPE，稀疏/低密度列，作者提出）

思路：结合频率（histogram）和经验累积分布函数（ECDF/CDF），把两者的不匹配度（频率差+累积分布差）作为 LSAP 的代价矩阵，求最小总代价的置换，从而匹配密文与明文。对低密度列也非常有效。

动机：对于非密集列，仅排序不够 → 结合 CDF + Histogram。

攻击构造：定义密文 & 明文的直方图： H_C, H_P

累积分布： F_C, F_P

构造代价： $\text{Cost}(i, j) = \alpha |H_C(i) - H_P(j)|^2 + (1 - \alpha) |F_C(i) - F_P(j)|^2$

效果：Admission Type: 70-100%; Length of Stay: 100%; 年龄: > 95%

（4）实验设计与实现细节

场景与数据：以电子病历数据库（EMR）为代表（理由：医疗数据敏感且常被作为 EDB 应用样例）。

目标数据：使用 2009 HCUP/NIS（National Inpatient Sample）中来自约 1050 家医院的记录；实验主要针对 200 个大型医院（也做 200 个小型医院的低密度试验）。

辅助数据：使用 2008 Texas PUDF 与 2004 HCUP/NIS（注意：目标/辅助年份与医院集合不同，模拟现实中辅助数据并非完全相同的情况）。

	Max	Min	Mean	SD
Large Hospitals	121,664	12,975	24,486	12,015
Small Hospitals	1,309	404	756	253

Table 1: Size of hospitals in number of patients

攻击目标列：sex, race, age, admission month, patient died, primary payer, length of stay, mortality risk, disease severity, major diagnostic category, admission type/source 等（见 Fig.1）。这些属性覆盖等值查询列与范围/排序列（因此对应 DTE 或 OPE）。

实现工具链：Parser（Python）：把目标/辅助数据解析为每个医院/属性的直方图与 CDF。Column Finder（Python）：在加密数据库中定位对应属性的密文字段（利用“不同值数量”这一泄露信息来匹配候选列）。Revealer（Matlab）：实现频率分析、 ℓ_2 -optimization、cumulative attack（Hungarian 算法求解 LSAP）。

作者报告每个医院每次攻击耗时极短（多数在毫秒级，复杂列如 **length-of-stay** 较慢），实现可在常见工作站上复现。

（5）关键实验结果

针对 DTE (ℓ_2 -optimization / frequency)：恢复 mortality risk 与 patient died 字段：对 $\geq 99\%$ 的 200 大型医院，能 100% 恢复这些患者的相关字段（即对每个病人的该字段恢复正确）。恢复 disease severity：在 $\geq 51\%$ 的大医院可以对 100% 的患者恢复正确。对 race、major diagnostic category、primary payer、admission source、admission type 等也能在大量医院恢复相当高比例。

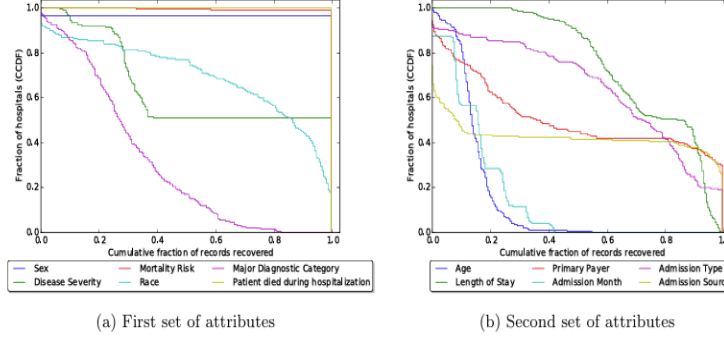


Figure 3: Results of ℓ_2 -optimization on DTE-encrypted columns on 200 largest hospitals with 2009 HCUP/NIS as target data and 2004 HCUP/NIS as auxiliary data

即使是取值范围很大的 **age** 与 **length-of-stay**，也能在大量医院恢复可观的记录比例（例如 **age**：对 $\geq 84.5\%$ 的医院能恢复 $\geq 10\%$ 的患者年龄；**length-of-stay** 对 50% 医院能恢复 $\geq 83\%$ 的患者。原因是真实分布高度集中）。

针对 OPE (sorting / cumulative)：sorting（对 dense 列）可在 dense 的属性上完全恢复（例如 disease severity、mortality risk 在大医院中 density=1 的比率很高，因而可完全恢复）。

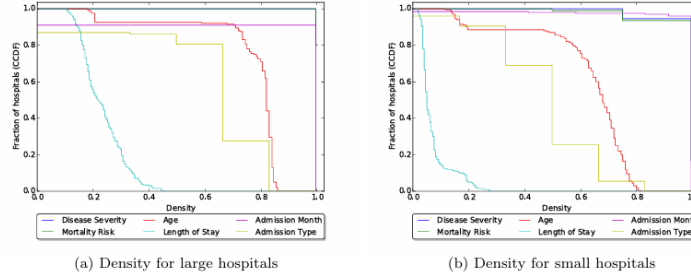


Figure 4: Density - Ratio of the number of values of an attribute present in a column to the total number of values of the attribute

cumulative attack（对低/中密度列）在 200 大医院上表现极好：在 $\geq 95\%$ 医院上能对 disease severity, mortality risk, age, length of stay, admission month, admission type 等恢复 $\geq 80\%$ 的患者；对 200 小医院上某些列（如 admission month、disease severity、mortality risk）在 $\geq 99.5\%$ 医院能 100% 恢复。

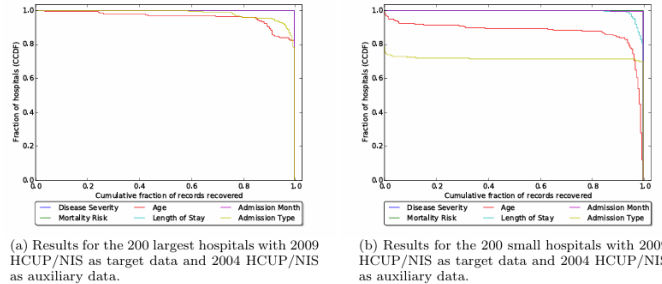


Figure 5: Results of Cumulative attack on OPE-encrypted columns

整体结论：在现实医疗数据场景下，PPE-based EDB（如 CryptDB）在 steady-state 下对这些推断攻击极脆弱。作者认为不应在 EMR 等敏感领域使用这类系统，且实验结果应被视作对 PPE-EDB 的下界（作者并未利用 query-leakage、也未攻击更弱的 join 层，实际风险可能更高）。

（6）作者结论

PPE（DTE/OPE）为执行性和兼容性付出了信息泄露代价；这些泄露结合公共/历史数据足以在大量现实场景下高准确率恢复敏感字段。对医疗 EDB，作者明确建议不要采用这类设计（至少不能以当前形式用于高度敏感数据）。

（7）对 CryptDB/PPE-based EDB 的安全性分析

为什么会被攻破（根本原因）：CryptDB 的“洋葱加密”会在 steady-state 下将某些列剥到 DTE 或 OPE，以支持等值或范围查询；而 DTE/OPE 本质上泄露了相等性或排序/秩信息，这些属性在现实数据中包含大量可利用的统计特征（频率分布、CDF、value-set 大小等），因此可被外部辅助数据/统计方法映射回明文。

对策方向（权衡安全↔性能）：不要在高敏感度数据（如 EMR）上使用 DTE/OPE；若一定要使用，明确承认风险并做额外缓解。

使用更强的构造：如 FHE/ORAM/secure enclaves（SGX）来减少结构泄露。但这些通常有性能或部署成本（或有额外信任假设）。CryptDB-like 设计的替代方案：交互式 OPE（泄露更少的顺序信息）或 ORE，但理论上仍然会有泄露；ORAM 可隐藏访问模式但昂贵。

最小化 PPE 的使用范围：仅对非敏感列或在导出统计时使用；对关键列使用 randomized / stateful encryption（避免可重复密文）。

引入噪声/分区/填充策略：在生成直方图/查询输出前对分布做掩码（类似差分隐私思想），或对列值做随机化/桶化以降低精确匹配的可行性。但这些会影响查询精度与性能。

减少 auxiliary-data 可用性与列标识泄露：CryptDB 的匿名化 schema、列值个数泄露、以及 onion 剥离过程均暴露信息——设计上应尽量隐藏列上“distinct count”或对 Column Finder 的信息进行混淆（例如加入 dummy 列，或对 distinct count 做模糊化/噪声）。论文里 Column Finder 利用 distinct-value 数量定位目标列是一个关键步骤。

监测与速率限制：检测异常的整体统计获取行为，限制对全库直方图/导出类操作的权限（但注意：论文在 ciphertext-only 情况下就能恢复，query-based 限制并不能完全阻止）。

（8）总结

CryptDB 通过“洋葱式加密（onion encryption）”结构实现了在不解密数据的前提下支持 SQL 查询的目标。其核心机制是在每个数据列外包裹多层加密，每层对应一种查询操作（如相等查询的确定性加密层 DTE、范围查询的序保持加密层 OPE 等），当应用需要执行相应查询时，系统逐步剥离外层密文，暴露满足功能要求的最弱层。然而，这种属性保留型加密（Property-Preserving Encryption, PPE）不可避免地泄露部分结构信息——如值之间的相等性与顺序关系。

从理论上讲，DTE 与 OPE 的安全性仅在极其受限的明文分布假设下才能保证，而在实际场景中，明文往往呈现明显的统计规律（如医疗数据中的年龄分布、疾病等级、支付类型等）。正因如此，《Inference Attacks on Property-Preserving Encrypted Databases》实验证明，即便在最保守的密文仅可见模型下

（ciphertext-only setting），攻击者仍能利用公开辅助数据与统计特征，通过频率

分析、最优匹配 (ℓ_2 -optimization) 或累积分布攻击 (cumulative attack) 等方法, 以极高的精度恢复被加密字段的真实值。在医疗场景中, 对部分字段 (如病情严重程度、死亡风险) 的恢复率甚至达到 99% 以上。这意味着 CryptDB 在实际部署后处于稳定运行状态 (steady state) 时, 其泄露的属性足以使攻击者重建大量敏感信息。

因此, CryptDB 的安全性问题并非源于密码算法本身, 而是其系统设计所固有的“功能-隐私权衡”导致: 为实现可查询性而牺牲了部分语义安全。攻击结果表明, PPE 层所暴露的模式信息在现实数据中具有极强的可区分性, 足以支撑精确推断。这种结构性泄露无法通过传统的密钥管理或访问控制修复, 必须在系统层重新设计, 例如引入更强的加密形式 (如可验证同态加密、ORAM 或安全执行环境) 或在查询输出中加入差分隐私机制以降低可识别性。

总体而言, CryptDB 的设计在性能与安全之间做出了大胆但危险的折中, 其在高敏感度数据场景 (如医疗、金融、政府数据库) 中已被证明无法提供足够的机密性保证。

3.2 对 CryptDB 的功能改进进行调研分析

阅读 CryptZip: Squeezing out the Redundancy in Homomorphically Encrypted Backup Data

(1) 研究主题与总体概述

文章聚焦于同态加密数据库的备份优化问题。在云计算与数据安全需求不断提升的背景下, 数据库往往采用加密存储以保护敏感信息。然而, 这种设计在带来安全性的同时, 也使得数据在备份与恢复阶段出现了极高的存储成本与性能开销。特别是像 CryptDB 这样的部分同态加密数据库系统, 为了在密文上直接执行 SQL 查询, 不可避免地对同一数据项生成多份加密副本, 以支持不同类型的操作 (例如相等比较、范围查询、求和等)。这类“多洋葱加密结构”虽然增强了系统功能性, 却导致数据库体积成倍膨胀。

论文作者提出的 CryptZip 系统, 旨在在不破坏安全性的前提下, 通过分析加密元数据, 自动识别出冗余加密列并进行选择性备份, 从而显著减少加密数据的备份存储成本。

CryptZip 的设计目标可以概括为两个方面: 一是显著降低加密数据库备份所需的存储空间, 二是在此基础上尽量缩短灾难恢复的时间开销。作者在设计中综合考虑了安全性、可恢复性和效率三者的平衡, 提出了可调节的备份策略框架, 使系统能在不同场景下灵活取舍。

(2) 研究背景与问题提出

随着云数据库广泛部署, 用户数据的安全威胁愈加严重。数据库管理员、云服务提供者甚至外部攻击者都可能通过各种方式非法访问后端存储。传统应对方法是对数据进行加密存储, 但这会阻断 SQL 查询的直接执行。为解决这一矛盾, MIT 提出了著名系统 CryptDB, 它通过在每个数据列外包裹多层加密 (称为 onion encryption) 来支持在密文上执行查询操作。每一层加密采用不同的部分同态加密算法 (例如确定性加密 DET、序保持加密 OPE、同态加法加密 HOM、可搜索加密 SEARCH 等), 系统根据查询需求动态剥离或启用相应的加密层。

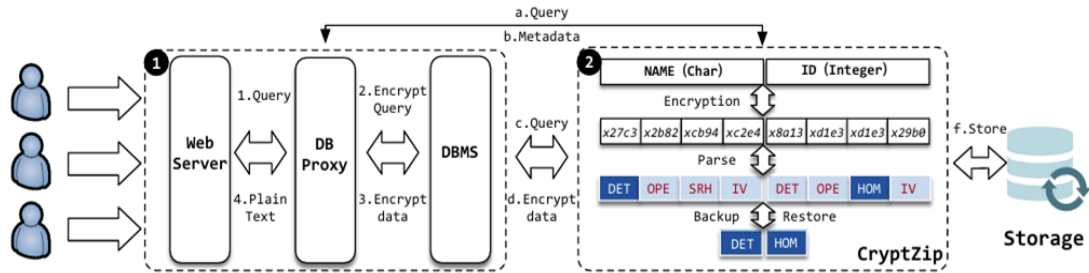
然而, 这种设计存在一个关键的副作用, **数据冗余严重**。为了保证所有类型

的查询都能执行，同一个明文字段往往被多次加密并分别存储为多个“洋葱副本”，导致数据库空间开销成倍增长。论文指出，在 TPC-C 基准测试下，CryptDB 数据的备份文件大小从明文的 79MB 增加到了 1.68GB，膨胀约 21.3 倍。更糟糕的是，由于加密密文的随机性极高，传统压缩算法（如 Gzip）几乎无法发挥作用，仅能减少约 15% 的空间。因此，在大规模云环境中维护周期性备份或跨数据中心冗余备份时，存储成本和网络带宽消耗都异常巨大。

作者因此提出核心研究问题：是否可以在不解密数据、不破坏安全性的前提下，有效去除加密数据备份中的冗余信息，从而显著降低存储开销？这一问题成为 CryptZip 系统设计的出发点。

（3）系统设计理念与总体结构

CryptZip 的基本思想是利用 CryptDB 系统的元数据信息（metadata）。在 CryptDB 架构中，代理层（Proxy）维护了每个数据库表的加密细节，包括各列使用的加密方案及对应的密文列名。作者发现，这些元数据实际上可以揭示出哪些列是由同一个明文字段复制加密而来，即所谓“语义冗余列”。基于这一观察，CryptZip 通过解析元数据来识别冗余关系，然后在备份阶段只选择其中一个或少数几个代表性加密列进行存储，从而达到“逻辑压缩”的效果。系统整体由三个主要部分构成：



元数据解析模块：从 CryptDB 代理获取每个表的加密映射关系，确定明文字段与其多洋葱加密副本之间的对应。

选择性备份模块：根据预设策略（空间优先、时间优先或平衡策略），选择某些加密列作为“恢复洋葱（recovery onion）”进行备份。

恢复模块：在灾难恢复时，通过解密备份的 recovery onion 并重新加密生成其他洋葱副本，完整恢复数据库。这种方法完全在密文与加密层级上操作，不需要访问明文数据库，也不改变 CryptDB 的查询逻辑，从而保证了系统的安全性与透明性。

（4）备份策略与理论模型

论文进一步提出了一个 Min-Space 模型来指导备份策略的制定。模型的目标是在给定恢复时间限制的前提下，使总存储空间开销最小化。设一个字段被多种洋葱加密后形成加密集 $OA = \{O_1, O_2, \dots\}$ ，其中每个洋葱的密文大小为 s_i ，加密与解密时间分别为 T_E, T_D ，并用布尔变量 d_i 表示是否可解密。作者建立了如下优化目标：

①空间开销： $SF = \sum_{O \in OS} S_O$

②恢复时间： $Ext(OS) = \min_{O \in OS} (T_D) + \sum_{O \in (OA - OS)} T_E$

③约束条件：至少选择一个可解密洋葱，即 $\sum d_O \geq 1$ 。

在此模型下，作者提出三种可供实际系统使用的备份策略：

①**Space-optimal（空间最优策略）：**只保留一个最小可解密洋葱，空间最

省但恢复时间最长。

②Balanced（平衡策略）：保留部分洋葱，在节省空间与恢复速度之间折中。

③Time-optimal（时间最优策略）：保留全部洋葱，不节省空间但恢复最快。

作者还指出，不同数据类型（整数、字符串）使用的加密算法差异明显。例如，字符串类型中只有 DET 可被解密，因此在字符串列的备份中必须保留 DET 层；而整数列的各洋葱（如 DET、OPE、HOM、ASHE）均可解密，因此存在更多组合选择。

（5）实验设计与评估方法

为了评估 CryptZip 的有效性，作者在两台高性能服务器上搭建了实验环境，系统配置包括 8 核 Intel Xeon E5-2630 v4 处理器、64GB 内存与 MySQL 5.7 数据库。测试使用两类基准：

微型基准（Micro-benchmarks）：包含一个整型表（100 万随机整数）和两个字符串表（长度分别为 16B 与 128B），用于测量加密层的加密时间、解密时间与压缩率。

TPC-C 基准：标准在线事务处理数据库，包含多种字段类型，用于综合评估系统在实际场景下的备份空间与恢复性能。

实验对比了多种方案：原始 MySQLDump 备份、Gzip 压缩后的加密备份、以及 CryptZip 在三种策略下的表现。测试重点关注两个指标：**存储空间成本（Storage Cost）**与**恢复时间（Recovery Time）**。

（6）实验结果与分析

在 Micro-benchmark 中，CryptZip 在不同数据类型和加密组合下均显示出显著的空间节省。

Table 4: Storage Cost and Recovery Time of Strategies for Integers

Backup Strategy	Strategies with HOM				Strategies with ASHE			
	Storage Cost	Storage Ratio	Recovery Time	Time Ratio	Storage Cost	Storage Ratio	Recovery Time	Time Ratio
MySQLDump	345MB	-	90s	-	65.9MB	-	19.9s	-
Space-Optimal Strategy	20MB	5%	+5866s	65x	19.5MB	30%	+132.8s	6.81x
Time-Optimal Strategy	297MB	86%	+16s	0.36x	44.7MB	68%	+12s	0.89x
Balanced Strategy	282MB	82%	+143s	2.45x	34.1MB	52%	+14.2s	0.8x

Table 5: Storage Cost and Recovery Time of Strategies for Strings

Backup Strategy	Strategies for String-16				Strategies for String-128			
	Storage Cost	Storage Ratio	Recovery Time	Time Ratio	Storage Cost	Storage Ratio	Recovery Time	Time Ratio
MySQLDump	59MB	-	17.9s	-	495MB	-	124s	-
Space-Optimal Strategy	16.2MB	27%	+94.3s	5.09x	143MB	29%	+316s	3.8x
Time-Optimal Strategy	42MB	71%	+6.4s	0.42x	415MB	84%	+17s	0.25x
Balanced Strategy	25.8MB	44%	+19.7s	1.17x	163MB	33%	+122s	1.48x

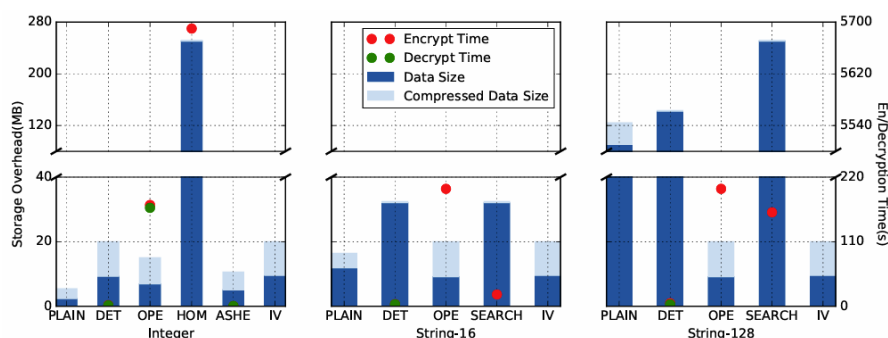
对于整型数据，采用 HOM 加密方案时，Space-optimal 策略将存储开销从 345MB 降至 20MB（仅为 5%），但恢复时间高达 5800 秒。Balanced 策略则能在存储减少 82%的同时，将额外恢复时间控制在 143 秒以内。采用 ASHE 加密方案时，Balanced 策略同样表现出良好平衡，存储减半、恢复时间减少 20%。

Table 6: Storage Cost and Recovery Time of TPC-C benchmark

Backup Strategy	Strategies with HOM				Strategies with ASHE			
	Storage Cost	Storage Ratio	Recovery Time	Time Ratio	Storage Cost	Storage Ratio	Recovery Time	Time Ratio
MySQLDump	1680MB	-	244s	-	572MB	-	91s	-
Space-Optimal Strategy	160MB	9.5%	+26215s	107.4x	160MB	27%	+3556s	39.8x
Time-Optimal Strategy	1368MB	81.4%	+246s	1.008x	533MB	93%	+180s	1.97x
Balanced Strategy	1205MB	71.7%	+1031s	4.23x	259MB	45%	+327s	3.59x

对于字符串数据（16B 与 128B），随着字符串长度增加，OPE 层的相对开销下降。Space-optimal 策略可节省 70%以上的存储空间，而 Balanced 策略在保

证恢复速度的情况下仍能减少约 55%的存储成本。



在 **TPC-C 综合测试**中，结果更加显著。普通明文数据库的备份文件大小为 79MB，而加密后达到 1680MB，使用 Gzip 压缩仍高达 1432MB。相比之下，CryptZip 的 Space-optimal 策略将其压缩到 160MB，节省高达 **90.5%**的空间；Balanced 策略在仅增加约 5 分钟恢复时间的前提下，仍能节省 **55%**的存储成本。这些结果充分证明了 CryptZip 在安全备份场景中的高效性和实用价值。

(7) 论文结论与未来工作

论文总结指出，CryptDB 通过多层洋葱加密实现了密文可查询性，但也因此带来了极高的冗余和存储膨胀问题。CryptZip 通过解析加密元数据，利用语义层冗余信息在逻辑层面实现了“压缩”，从而在不破坏安全性的条件下显著降低了备份空间。系统提出的 Min-space 分析模型为加密数据库的备份策略提供了量化依据。实验证明，CryptZip 在 TPC-C 基准下可实现 70%–90.5%的存储节省。

当前的恢复过程需要对备份数据进行解密并重新加密生成各层洋葱，这可能引入潜在安全风险。未来计划结合硬件可信执行环境(TEE)技术，如 Intel SGX 或 AMD Memory Encryption，将恢复过程封装于硬件安全边界内，从而进一步增强系统的安全性与可信性。

(8) 对 CryptDB 功能改进的分析

从系统架构角度看，CryptZip 对 CryptDB 的主要改进集中在**存储与运维层**。首先，它通过引入“元数据感知”的备份机制，使 CryptDB 能够在加密状态下进行智能化去冗余存储，这是原系统所缺乏的能力。其次，CryptZip 赋予数据库管理员可调节的备份策略接口，用户可以根据业务场景选择空间优先或恢复优先方案，实现灵活的性能权衡。再次，CryptZip 改善了加密数据的可压缩性，使加密数据库在云端部署与备份时更具经济性。更重要的是，该方案保持了与原有 CryptDB 架构的兼容性，无需修改查询代理或数据库引擎，因此易于集成与扩展。

未来的改进方向包括：在恢复阶段引入硬件安全模块以避免明文暴露；结合差分隐私或噪声注入技术以进一步提升安全性；以及将元数据驱动的去冗余理念扩展到增量备份与跨版本同步场景。

总体而言，CryptZip 的提出不仅优化了 CryptDB 的存储效率，也为加密数据库的系统性设计提供了新的思路，即在保持安全的前提下，通过结构层信息挖掘提升系统资源利用率。

3.3 对同类系统调研分析

Opaque 是由 Wenting Zheng 等人提出的一个面向分布式大数据分析的机密计算平台，其论文题为“Opaque: An Oblivious and Encrypted Distributed Analytics Platform”，并在 NSDI/OSDI 系列会议上发表。Opaque 的目标是让用户能够把

敏感数据加密并上传到不受信任的云集群上，同时在云端以接近原生 Spark SQL 的方式运行复杂分析查询，但又不将明文数据或访问模式泄露给云平台或管理员。为此，Opaque 将数据加密、计算完整性校验与“oblivious（不泄露访问模式）”的分布式算子结合起来，构成一个可扩展的、能够支持关系算子的机密分析平台。

在动机层面，Opaque 关注两类典型威胁：一是云平台或操作系统级别的恶意或被攻破组件会直接读取内存或持久化的数据；二是即便数据在安全执行环境（如 Intel SGX）中被保护，数据访问模式（access patterns）仍可能通过外部可观察的内存/IO 行为被泄露（例如通过页面/缓存/IO 监测推断出查询的热点、排序或筛选条件）。Opaque 的出发点是仅靠把代码放进 TEE 并不能阻止访问模式泄露，必须在算子设计层面提供“obliviousness”来阻断基于访问模式的推断攻击。论文对威胁模型与保护目标做了清晰定义：保护数据的机密性、保护查询的访问模式（即保证 obliviousness），并提供计算结果的完整性验证。

在架构与实现上，Opaque 基于 Spark SQL 做了有限改造：它保留了 Spark 的执行模型（RDD/分区等），但在数据读入、Shuffle、Join、GroupBy、排序等关键阶段替换或增强了算子，使用**分布式的 oblivious 算子（例如 oblivious shuffle / oblivious global sort / oblivious join）**来隐藏跨节点与跨分区的访问模式。为了把可扩展性和机密性结合起来，Opaque 将实际的密文数据与代码一起放入 SGX enclaves 中执行，但对外部的 I/O 和内存访问采用确定清洗（deterministic padding、dummy records、fixed-size transfers）等技巧，确保外界无法从通信量、请求序列或磁盘/内存访问显著区分真实数据的结构。实现上，Opaque 在 Spark 上做了“最少侵入”修改，使得用户可以仍然用熟悉的 SQL API 提交查询，而平台在后台替换为安全的执行计划。

Opaque 的核心技术贡献可分为三类：一是分布式 oblivious 算子库（包括 oblivious repartition/oblivious global sorting/oblivious join 等），这些算子保证在外部可观测的行为上（如每个阶段的 I/O 模式与网络流量）对不同输入产生相同的外部特征；二是查询规划与优化，即如何在可接受代价内把一个给定的 SQL 查询分解为一系列 oblivious 与非-oblivious 算子的混合计划，从而在性能与隐私之间做工程化折中；三是系统集成与工程实践。把 SGX enclave、网络协议、数据加密与 Spark 的执行引擎整合，使系统在真实集群上可运行并评估。论文提供了这些算子的设计细节（例如基于 ORAM 风格的局部 buffering+阶段性交换来实现 oblivious shuffle）并讨论了代价模型。

在性能与安全权衡方面，Opaque 的测评展示了“有代价，但可接受”的现实：与未加保护的 Spark SQL 相比，Opaque 在不开启 obliviousness 时仅引入有限开销（在某些工作负载下甚至接近原生），这是得益于 SGX 在 CPU 内部提供的高速隔离执行；当开启完整的 oblivious 模式以保护访问模式时，代价显著增加（论文中给出的范围是从约 $1.6\times$ 到几十倍的开销，取决于查询类型和数据规模）。作者还展示了在某些常见分析任务上，obliviousness 的额外成本可以被系统级优化（如重用 oblivious 排序、合并多个操作的单次 oblivious shuffle）所部分抵消。总体结论是：要在云端做到“机密且不泄露访问模式”的大规模分析，使用 TEE+分布式 oblivious 算子是一个可行但成本可观的工程方案。

Opaque 的局限性与实际挑战也非常值得注意并在论文中被坦率讨论。首先，它依赖于硬件 TEE（主要是 Intel SGX）作为可信根，这带来了两个问题：SGX 的可用内存（enclave 内存）有限，频繁的 enclave $\leftrightarrow$ 非-enclave 切换本身会带来性能惩罚，而且 SGX 曾被证明存在侧信道（如缓存/分支/页面泄露）以及微架构

攻击面的脆弱性；其次，完全的 obliviousness 在许多数据密集型操作（尤其是全局排序和大规模 join）上代价非常高，会大幅增加网络与计算开销，从而影响可伸缩性与成本效益。论文和后续社区工作均强调：工程上需要权衡何时必须保证 oblivious（例如在多方联合或高敏感度场景），何时可以放松以换取性能。

在学术与产业影响方面，Opaque 被视为将 TEE 与 oblivious 算子结合以实现可扩展机密分析的“开创”工作之一；它催生了后续大量研究（例如针对更快的 oblivious primitives、改进的 ORAM 分布式实现、以及将差分隐私与 TEE 结合的方案），并且激发了产业界将机密计算作为产品化方向（例如 Opaque 的研究成果后续被商业化，出现了以“Opaque/OPAQUE”命名的公司/产品，提供 Confidential AI 与隐私数据协作平台，已与云厂商与企业客户展开合作）。这些进展表明，从研究样机到工程化产品，机密计算与隐私分析正在进入实际采纳阶段，但技术可信度、合规性与运营复杂度仍是推广路上的关键门槛。

对比 CryptDB：Opaque 提供了一个系统性且更强的保密目标，CryptDB 的策略是通过部分同态或 PPE（property-preserving encryption）在密文上保持某些查询能力，结果不可避免地泄露等价/排序等信息，从而在静态数据场景下容易被辅助信息攻击恢复明文；而 Opaque 追求的是在执行过程中既不泄露明文，也不泄露访问模式（通过在硬件 enclaves 内执行并使用 oblivious 算子隐藏访问模式），因此理论上能抵抗包括访问模式推断在内的更强攻击类。不过，这种额外的安全性是以更复杂的实现、性能开销和对硬件可信度的依赖为代价的。换言之：CryptDB 在查询功能与性能上更“轻量”，但面临的统计/推断风险更高；Opaque 在安全性上更强，但工程代价、硬件假设与操作复杂度也更高。

对 CryptDB 的可借鉴改进建议（基于 Opaque 的思想与实践）有几条值得在工程/研究上优先考虑：

在关键敏感操作路径引入受信任执行环境：对那些高度敏感列或计算（例如身份识别、合并后的关键汇总），可以将代理或特定计算放到 TEE 中执行，从而避免将这些明文或详细访问模式暴露给数据库主机。

混合使用 oblivious 与非-oblivious 算子：并非所有查询都需要完全 oblivious；可以设计策略判断哪些查询/阶段必须 oblivious（如跨租户 join），哪些可以走低成本 PPE 路径，这类似 Opaque 的 query planner 思路。

增量引入 oblivious primitives：先实现代价相对较低的 oblivious 局部算子（如局部 padding 的 partition）来减少关键路径的访问模式泄露，再在需要时扩展到全局 oblivious 操作。

在恢复与备份环节使用 TEEs：像 CryptZip 那类需要在恢复时解密并重新加密的操作，可以放在硬件隔离环境中运行以减少中间明文泄露风险。

评估并缓解硬件侧信道：因为 Opaque 与任何 SGX 方案一样可能遭遇侧信道，任何把关键路径放 TE 的改进都应同时采用 side-channel 缓解（例如 constant-time 实现、页面访问模式平滑、噪声注入或 enclave 内部 secure paging 设计）。这些做法可逐步提升 CryptDB 在现实部署中的抗推断能力，同时尽量控制性能损失。

最后，关于 Opaque 的现实意义与未来方向：Opaque 展现了把高级数据库功能（SQL/Analytics）和强机密性保障结合起来的可行路径，并推动了“Confidential Computing”从研究向产品化的迁移。随着云厂商、芯片厂商以及行业合规需求的共同推动，像 Opaque 这类设计将继续演进：要么降低 oblivious 算子的成本（通过新的算法与系统优化），要么通过更强大的硬件（更大的 enclave、

硬件支持的 oblivious primitives) 来缩短性能差距。同时, 如何把差分隐私、可证明安全与合规审计整合进这样的系统, 也是当前研究社区与工业界关注的重点。对任何考虑在云端处理敏感数据的组织, Opaque 提供了一条清晰的参考路径: 以硬件为根、在算子级别保证不可观测性、并在查询层面精细化安全与性能的折中。

3.4 设计一个面向安全与性能权衡的 CryptDB 改进方案

在对 CryptDB^{[1][3]}架构、属性保持加密(PPE)的泄露风险以及 CryptZip^[8]等改进工作进行综合调研后, 我提出一个面向工程可落地的改进方案

Leakage-aware CryptDB++。该方案的目标并非完全重构 CryptDB, 而是在其原有代理-数据库结构上, 以较低侵入性方式增强安全性和备份性能, 使系统在真实部署场景中更加稳健与易用。

(1) 设计动机: 现有 CryptDB 的两个突出问题促使我们提出改进:

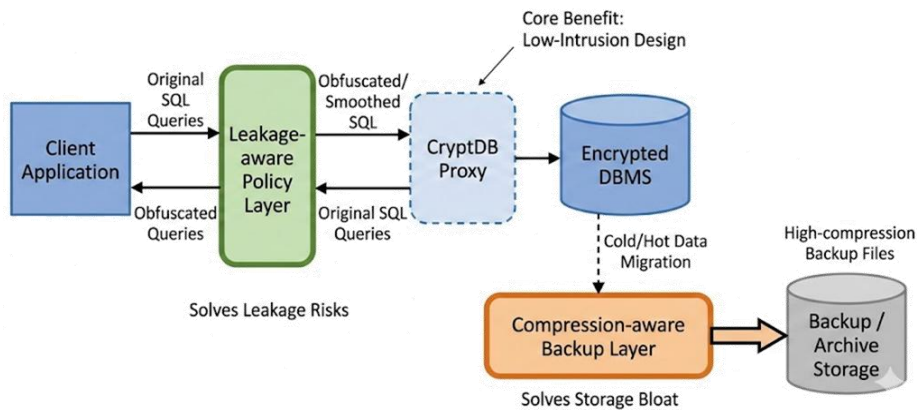
a.属性保持加密的结构泄露: CryptDB 使用 DET、OPE、HOM 等加密方式以支持 SQL 查询, 但不可避免地泄露: 等值模式、顺序与范围结构、长期访问模式。

已有研究表明, 仅通过密文分布、排序关系以及少量辅助背景信息, 攻击者即可对 PPE 保护的数据发起高精度推断攻击^[3]。特别是 Naveed 等人在 CCS 2015 中展示了针对 DET 与 OPE 的系统性推断方法, 可在真实数据集上恢复大部分原始数据的统计特征, 从而对用户隐私造成严重威胁。此外, 访问模式与查询频率本身也被证明是重要的泄露来源^{[4][7]}。

b.加密备份体积膨胀: 密文本身高度随机、难以压缩, 使加密数据库备份体积远大于明文数据库^{[8][9]}。对于周期性备份的大型系统, 这会造成明显存储成本压力。

因此, 我们需要一个兼顾泄露控制与存储友好性的综合增强方案。

(2) 整体思路: Leakage-aware CryptDB++从 CryptDB 的核心优势出发, 并在其两侧增设两大增强模块:



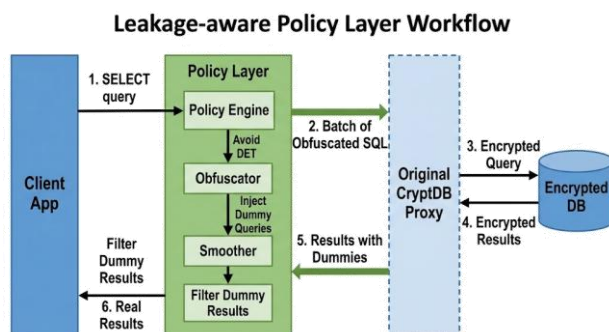
①**泄露感知策略层 (Leakage-aware Policy Layer)**: 位于应用端与 CryptDB proxy 之间; 用于自动选择合适的加密方式、模糊访问行为、平滑频率分布。

②**压缩友好备份层 (Compression-aware Backup Layer)**: 位于存储端和备份工具之间, 将冷数据迁移到压缩友好型编码结构中, 提高备份压缩比。

这两个模块互不影响 CryptDB 原有的 onion 加密与 SQL 功能, 使方案具有较强的可插拔性。

(3) 模块设计

a.泄露感知策略层：该模块通过列级策略+查询模糊化+轻量频率扰动减少 DET/OPE 的统计结构泄露^[7]。



①列级安全策略选择

系统根据 schema 分析每一列的：敏感度（如：用户隐私类/业务类/公开类）、查询需求（等值、范围、聚合）

自动选择更安全的加密方式，例如：高敏感+等值查询：避免 DET，使用随机化哈希+等值索引^[2]；范围查询列：采用 safer-ORE 或基于分桶的区间编码替代 OPE^{[5][6]}；低敏感列：保留原方案以避免性能损失。这样既降低泄露，又不让整个系统一刀切地牺牲性能。

②查询模糊化（Query Obfuscation）

为减弱访问模式泄露^[4]，策略层引入以下机制：

Batching：合并短时间内重复查询，减弱访问频率泄露

Dummy Queries：按比例注入对随机/dummy 数据的查询，增加访问分布噪声

结果端过滤：代理负责剔除 dummy 查询返回的数据，保持应用透明。

攻击者很难通过日志判断哪些记录是真正的热点。

③频率平滑（Frequency Smoothing）

针对 DET 密文的频率分布，采取简单但有效的扰动策略：

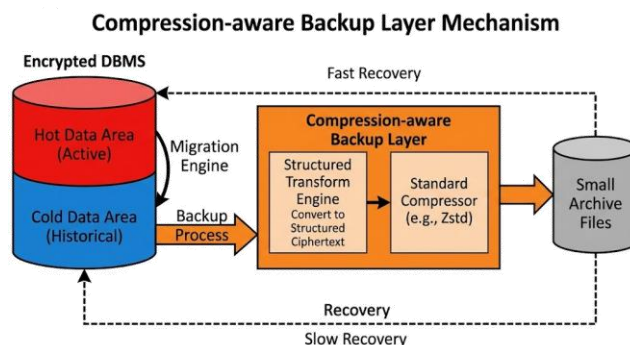
高频值→拆分成多个编码（如 1 → 1a/1b/1c）

低频值→自动添加极少量 dummy 行，使分布更平滑

这样可以有效降低基于密文统计分布的重构攻击成功率^[10]。

b.压缩友好备份层

为解决密文备份膨胀问题，我引入冷热数据分层与压缩友好型编码。



①热/冷数据分离：热数据区（近期频繁访问）保持 CryptDB 原有结构，确保查询速度；冷数据区采用结构化密文编码，使相同字段的密文更具“可压缩性”。这本质上是对 CryptZip 思路的扩展，将其变成 CryptDB 的一部分。

②**压缩友好加密+传统压缩**: 冷区数据采用结构化密文后, 对 Gzip、Zstd 等压缩算法的记录间模式更友好, 可使备份体积显著下降^{[8][9]}。

③**分级恢复机制**: 恢复时先恢复热区, 保证主要业务尽快可用; 再按需异步恢复冷区, 提高灾难恢复效率。

(4) **预期效果与价值**: Leakage-aware CryptDB++ 并未追求使用昂贵的 ORAM/TEE 等机制, 而是通过更轻量化的方式实现以下收益。

a. 安全性提升: 推断攻击成功率显著下降、高频/低频值难以识别、访问热度无法直接观察、整体泄露面更难利用。这提高了 CryptDB 在更广泛场景中的可信度。

b. 存储与备份成本降低: 冷数据备份可实现明显压缩, 适用于: 长期日志、历史订单、不常查询的归档数据。

在大规模数据库中, 这一改进具有实际工程意义。

c. 保持高可用性与低侵入性: 应用端几乎无需修改, CryptDB 本身也无需重写核心机制, 增强模块可以在实际系统中逐步部署。

Leakage-aware CryptDB++ 的核心贡献在于: 以策略层+存储层的方式增强 CryptDB, 而非推翻其架构; 用工程可落地的轻量化方法控制泄露风险; 显著降低加密备份的空间开销; 在性能、存储与安全之间找到更灵活的平衡点。

参考文献:

- [1] Popa, R. A., Redfield, C. M. S., Zeldovich, N., and Balakrishnan, H. **CryptDB: Protecting Confidentiality with Encrypted Query Processing**. In *Proceedings of the 23rd ACM Symposium on Operating Systems Principles (SOSP)*, 2011.
<https://people.csail.mit.edu/nickolai/papers/raluca-cryptdb.pdf>
- [2] Popa, R. A., and Zeldovich, N. **Multi-Key Searchable Encryption**. In *Proceedings of the 22nd USENIX Security Symposium*, 2013.
<https://eprint.iacr.org/2013/508.pdf#:~:text=We%20construct%20a%20searchable%20encryption%20scheme%20that%20enables,in%20client-server%20applications%20against%20attacks%20on%20the%20server.>
- [3] Naveed, M., Kamara, S., and Wright, C. V. **Inference Attacks on Property-Preserving Encrypted Databases**. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015.
- [4] Islam, M. S., Kuzu, M., and Kantarcioglu, M. **Access Pattern Disclosure on Searchable Encryption**. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2012. <https://personal.utdallas.edu/~mxk055100/publications/ndss2012.pdf>
- [5] Boldyreva, A., Chenette, N., Lee, Y., and O'Neill, A. **Order-Preserving Symmetric Encryption**. In *Advances in Cryptology – EUROCRYPT*, 2009.
<https://faculty.cc.gatech.edu/~aboldyre/papers/bclo.pdf>
- [6] Lewi, K., and Wu, D. J. **Order-Revealing Encryption: New Constructions, Applications, and Lower Bounds**. In *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016. <https://eprint.iacr.org/2016/612.pdf>
- [7] Cash, D., Grubbs, P., Perry, J., and Ristenpart, T. **Leakage-Abuse Attacks Against Searchable Encryption**. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2015. <https://dl.acm.org/doi/pdf/10.1145/2810103.2813700>
- [8] Chen, S., Li, S., Yiu, S. M., and Wang, P. **CryptZip: Compression-Friendly Encrypted Data**

Storage. In *Proceedings of the IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2015.

<https://acs.ict.ac.cn/baoyg/pub/202203/P020220317732938145367.pdf#:~:text=We%20present%20CryptZip%2C%20a%20backup%20and%20re-covery%20system,one%20or%20several%20columns%20among%20semantically%20redun-dant%20columns.>

[9] Bellare, M., Boldyreva, A., and O’Neill, A. **Deterministic and Efficiently Searchable Encryption.** In *Advances in Cryptology – CRYPTO*, 2007. <https://eprint.iacr.org/2006/186.pdf>

[10] Grubbs, P., Lacharité, M.-S., Minaud, B., and Paterson, K. G. **Learning to Reconstruct: Statistical Learning Theory and Encrypted Database Attacks.** In *Proceedings of the 2019 IEEE Symposium on Security and Privacy (SP)*, pp. 1067–1083.

<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8835288>